

SISTEMAS ELETRÓNICOS

ENG. ELETROTÉCNICA E DE COMPUTADORES

3º trabalho laboratorial

Beatriz Coelho ist 1107113
Carolina Baltazar ist 1107117
Lúcia Gomes ist 1102183

2º Semestre 2024/2025 – P3

Introdução

Este trabalho tem como objetivo o desenvolvimento de um software para implementação do μ Oscilloscope baseado num sistema embebido, capaz de realizar a aquisição, processamento e visualização de sinais elétricos. Para tal, foi implementado um sistema que utiliza um ecrã TFT e um microcontrolador para recolha e análise dos sinais, permitindo a exibição das formas de onda e a aplicação de processamento digital de sinal, incluindo filtragem e análise espectral.

O projeto tem ainda funcionalidades adicionais, como a seleção de escalas de tempo, tensão e frequência, envio de medições por e-mail e exibição de valores estatísticos do sinal, o que permite a análise de sinais em tempo real.

Este relatório apresenta o nosso desenvolvimento do software, a implementação prática e a análise dos resultados obtidos.

Análise do Programa

Esta parte do relatório apresenta a análise da implementação do programa principal (`main.py`) do projeto μ Oscilloscope, desenvolvido em Python para correr num sistema embebido ESP32. Avalia-se a conformidade com o enunciado oficial, com destaque para funcionalidades implementadas, testes realizados e pontos a melhorar.

1. Inicialização e Visualização

Ao iniciar, o display apresenta corretamente uma grelha com 10 divisões horizontais e 6 verticais, utilizando toda a área exceto os 16 píxeis inferiores, onde são mostradas as escalas. As escalas verticais (1, 2, 5, 10 V/div) e horizontais (5, 10, 20, 50 ms/div) estão implementadas e visíveis. O ícone de Wi-Fi é também corretamente apresentado com a função `set_wifi_icon`. Quando carregado no botão 11 reinicia a grelha e reamostra a uma escala de 2V/div e 5ms/div.

2. Leitura e Conversão dos Dados

A função `ler_amstras()` recolhe 240 amostras do ADC conforme a escala de tempo selecionada. Os valores lidos são convertidos em tensão através de uma função calibrada `adc_para_volt()`, usando a fórmula:

$$V = 0.01265 \times \text{ADC} - 26.25898$$

Esta fórmula resulta de uma calibração feita com base no módulo utilizado.

3. Visualização da Forma de Onda

A forma de onda adquirida é convertida para coordenadas em píxeis e representada graficamente com a função `display_nline()`. A mudança de escala permite ajustar manualmente a visualização através dos botões 21 (verticalmente) e 22 (horizontalmente). Estão disponíveis quatro escalas verticais (0.5, 1, 2.5, 5 V/div) e quatro horizontais (240, 120, 60, 24 Hz/div) para DFT e, para, uma mostragem normal, (5, 10, 20, 50) para uma escala vertical e (1, 2, 5, 10) para uma escala horizontal.

4. Medições Automáticas

O programa calcula automaticamente os valores de **Vmax**, **Vmin**, **Vavg** e **Vrms**, apresentando-os no display com precisão de duas casas decimais.

5. Envio de Email

A funcionalidade de envio de email (`enviar_email()`) permite partilhar os dados do sinal e as medições, utilizando o método `tft.send_mail()`, conforme especificado no enunciado. O botão 12 envia o email.

6. Modo Espectro (Transformada de Fourier Discreta)

A função `mostrar_espectro_dft()` implementa a DFT manualmente, calcula os módulos da transformada e converte-os em valores representáveis no display através dos botões 23.

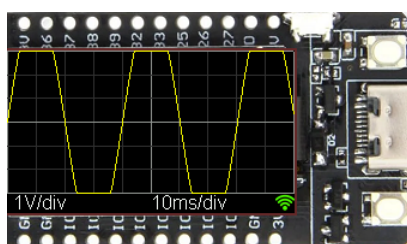
7. Filtragem Digital

A filtragem digital é feita com um filtro **Chebyshev IIR** de 1ª ordem com frequência central $f_0 = 100$ Hz e largura de banda $BW = 50$ Hz. O filtro é aplicado com base numa fórmula da diferença obtida por transformação bilinear, na função `filtro_chebyshev()` cujo é obtido através do botão 13.

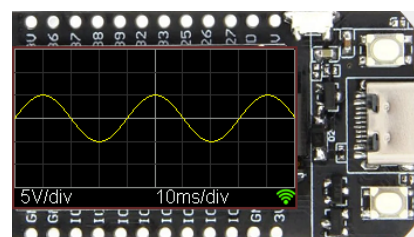
Apesar de grande parte dos requisitos terem sido cumpridos, existem algumas funcionalidades previstas no enunciado que não foram implementadas na versão final do programa: O código não ajusta automaticamente a escala vertical para melhor representar o sinal após a filtragem (como seria pedido ao premir o botão 13). Em vez disso, mantém a escala selecionada anteriormente pelo utilizador. A função `mostrar_espectro()` infelizmente está feita para uma escala de 120 invés de 240. O motivo foi porque se a escala fosse alterada para 240 o ecrã não ficava centrado e parte do *display* desaparecia.

Testes no simulador De forma a testar o código sem utilizar o módulo IoT, foram feitos testes no simulador fornecido pelos docentes.

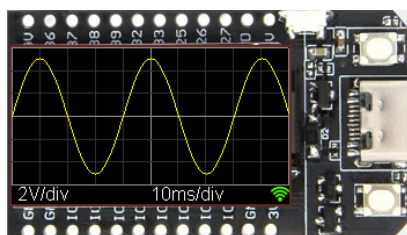
Para a funcionalidade 11, com uma onda sinusoidal de 25Hz, com amplitude 5V e sem offset DC. Alteraámos as escalas, dos Volts e tempo/freq. Nota: Aqui apenas estão representadas duas escalas diferentes, para não sobrecarregar o trabalho com muitas imagens, no entanto as outras escalas estão a funcionar normalmente



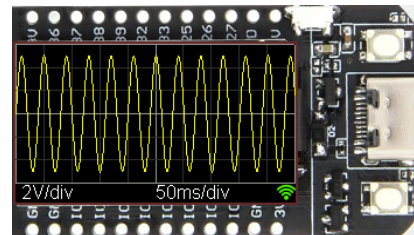
(a) 1V/div e 10ms/div



(b) 5V/div e 10ms/div



(a) 2V/div e 10ms/div



(b) 2V/div e 50ms/div

Para testar o comando 23, analisar o espectro, criamos uma onda quadrada de 60Hz, com 4V de amplitude e tensão DC de -1V;

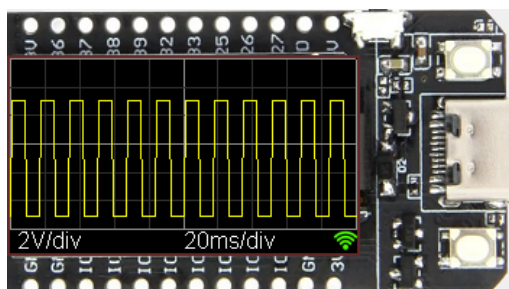


Figura 3: Onda quadrada

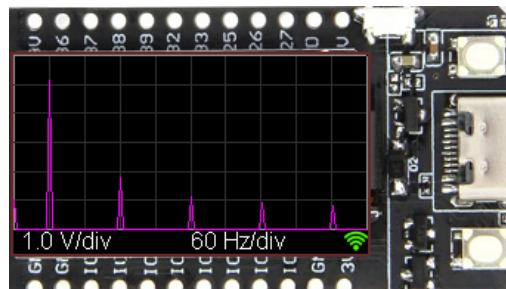


Figura 4: Espectro da onda quadrada

Tensões DC

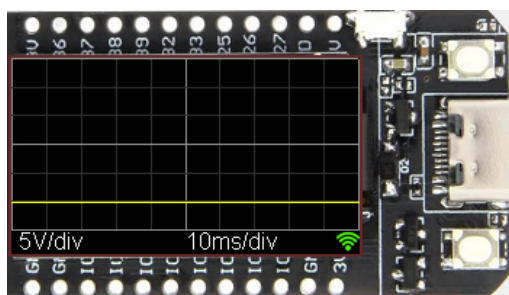


Figura 5: -10V

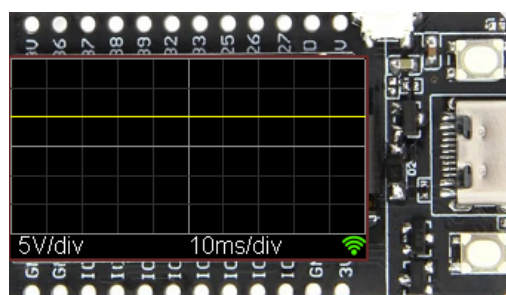


Figura 6: 5V

Com o comando 13, apresentamos a forma de onda antiga, após a filtragem passa-banda do tipo IIR. Para uma onda quadrada de 100k, para uma onda sinusoidal de à frequência central do filtro 100Hz nos extremos da banda de passagem (aproximadamente 80Hz e 130Hz) e a uma frequência da banda de atenuação (500Hz);

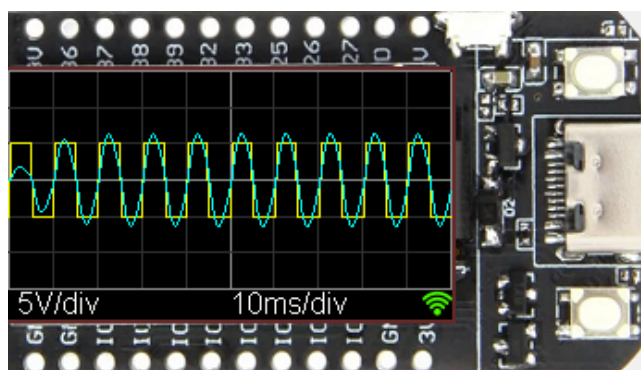
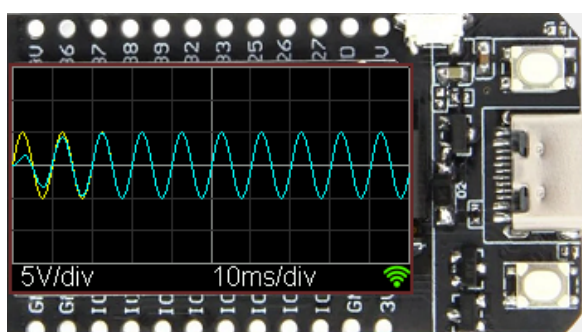
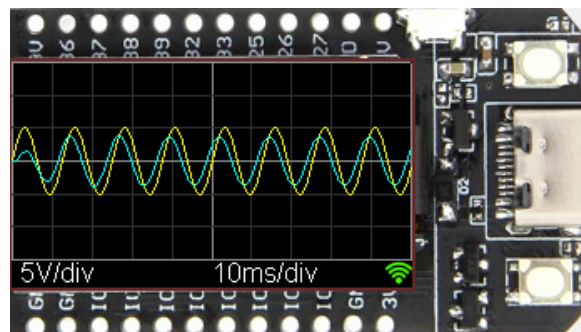
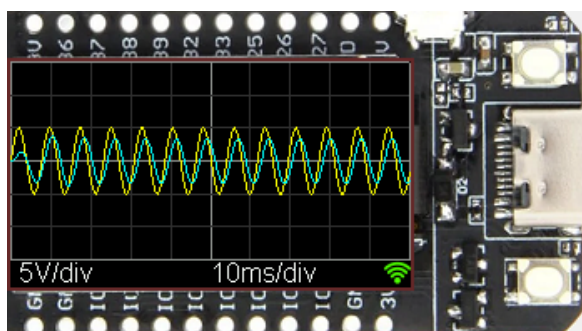
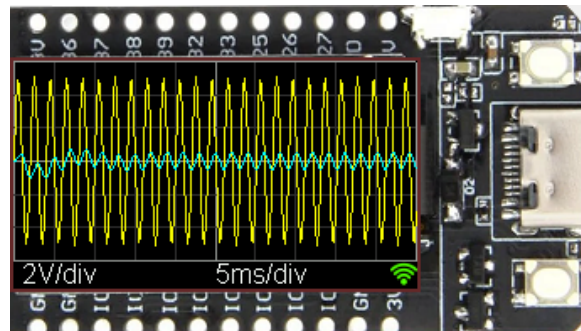


Figura 7: Onda quadrada de 100kHz depois de passar no filtro

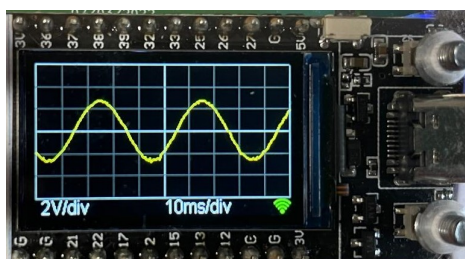
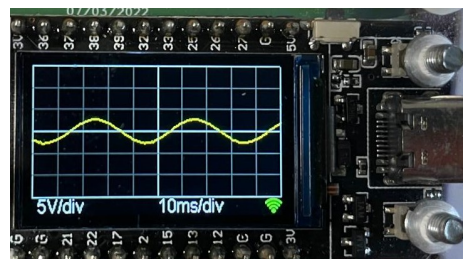
(a) $f = 100$ (b) $f = 80$ **Figura 8:** Comparação entre $f = 100$ e $f = 80$ (a) $f = 130$ (b) $f = 500$ **Figura 9:** Comparação entre $f = 130$ e $f = 500$

Testes no laboratório

Fizemos os mesmos testes realizados no simulador no laboratório

Sem Calibração

Para os testes sem calibração usamos uma onda com amplitude de 2.5V, Mais uma vez so vamos apresentar duas ecalas, sendo que as outras fucnionam corretamente.

**Figura 10:** 2V/div e 10ms/div**Figura 11:** 5V/div e 10ms/div

Mudar o tempo

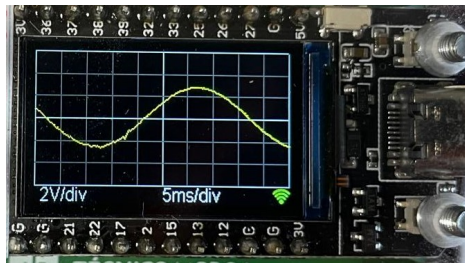


Figura 12: 5ms

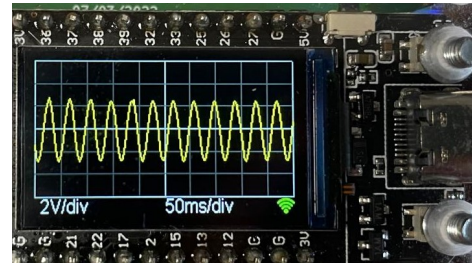


Figura 13: 50ms

Espectros

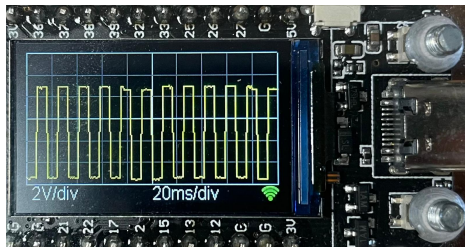


Figura 14: Enter Caption 1

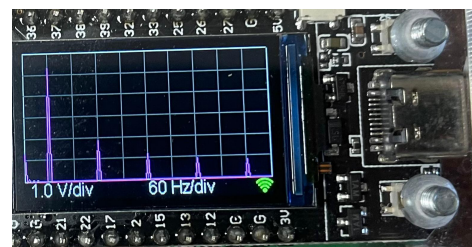


Figura 15: Enter Caption 2

Filtro sem calibração Não mostramos os resultados de todas as funções pedidas, uma vez que foram semelhantes para os outros casos.

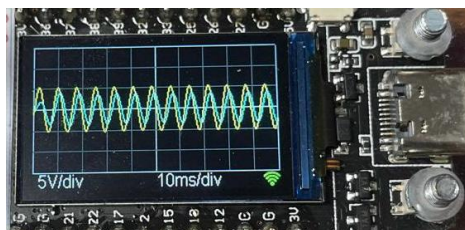


Figura 16: Filtro 130 Hz

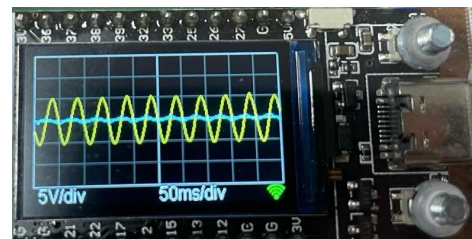


Figura 17: Filtro 500 Hz

Calibração

Para realizar os mesmos testes, mas com calibração, utilizamos o gerador de funções com uma frequência aproximadamente nula e amplitude AC também. Depois, fomos alterando os valores do offset, valores DC pré-definidos - ($-10V$, $-5V$, $0V$, $+5V$, $+10V$), para ter mais precisão, medimos os valores DC com um multímetro. Utilizando o código fornecido pelos docentes `main_exemplo.2.py`, encontramos para cada ponto, dos valores DC, a média de 100 amostras do conversor analógico digital. Assim, com os pontos definidos, realizamos o gráfico apresentado na figura x, e a respetiva equação.

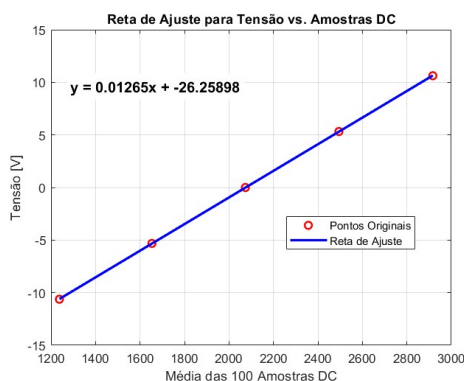


Figura 18: Gráfico para encontrar a reta de ajuste para a calibração do modulo IoT

Com esta equação, conseguimos alterar a função `def adc_para_volt(adc)` do nosso código `main_exemplo_2.py`, para corresponder à nova equação da reta de ajuste, de notar que agora não subtraímos 1 e dividimos pelo fator $\frac{1}{29.3}$, uma vez que a equação que encontramos não precisa desse ajuste - o gráfico que encontramos é contrário ao do enunciado.

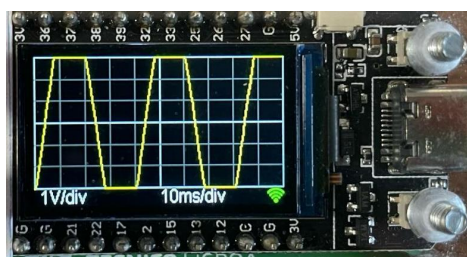


Figura 19: 1V/div e 10ms/div

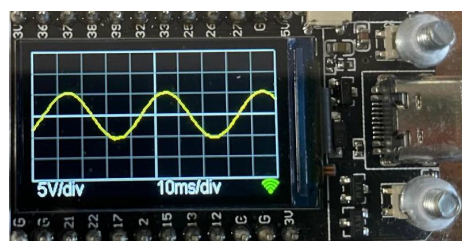


Figura 20: 5V/div e 10ms/div

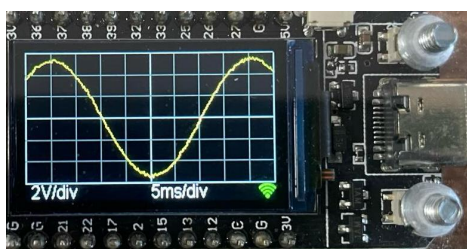


Figura 21: 5ms/div e 2V/div

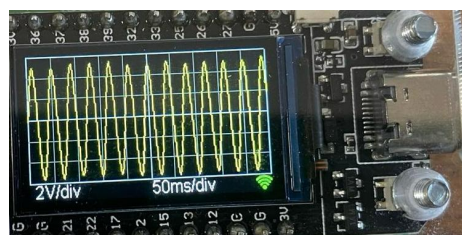


Figura 22: 50ms/div e 2V/div

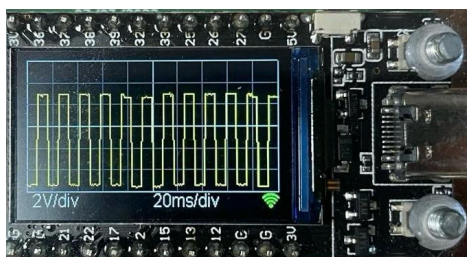


Figura 23: onda para espectro

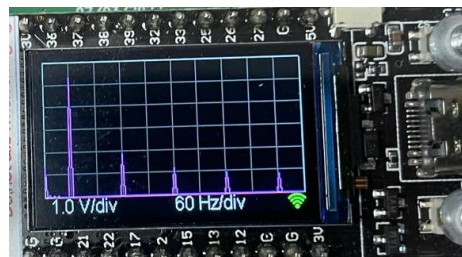
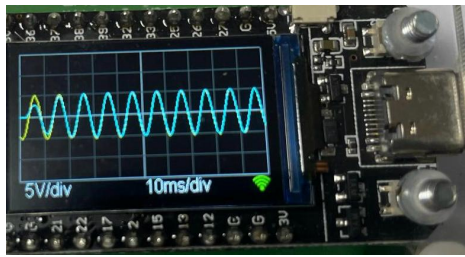
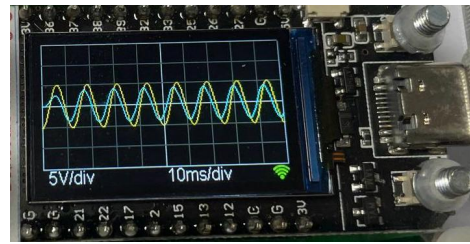
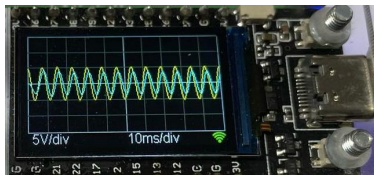
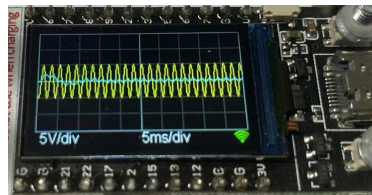
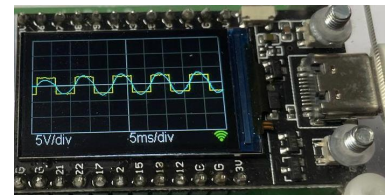


Figura 24: Espectro da onda quadrada

**Figura 25:** filtro 100 Hz**Figura 26:** filtro 80 Hz**Figura 27:** Filtro 130 Hz**Figura 28:** Filtro 500 Hz**Figura 29:** Filtro quadrada

Após a realização dos testes laboratoriais, foi possível comparar as medidas obtidas no laboratório antes e depois da calibração. Observou-se uma melhoria na precisão dos resultados obtidos, com os valores medidos a aproximarem-se dos valores da simulação após o processo de calibração.

Antes da calibração, os resultados apresentavam pequenos desvios, que apesar de não serem excessivamente grandes, poderiam comprometer a fiabilidade das medições. Estes desvios podem ter origem em vários fatores, como o desgaste natural dos componentes, condições ambientais, ou variações internas do sistema.

Após a calibração, notou-se um alinhamento entre os valores medidos e os teóricos, demonstrando que a calibração teve impacto direto na melhoria da exatidão do sistema de medição.

Esta comparação evidencia a importância da calibração como uma etapa essencial nas experiências laboratoriais.

Conclusão

Ao longo deste trabalho foi desenvolvido e testado um sistema embebido com capacidade para funcionar como osciloscópio digital, utilizando um microcontrolador para aquisição e visualização de sinais analógicos. A implementação incluiu o desenvolvimento de uma interface gráfica no computador para visualização em tempo real das formas de onda.

Foram realizados testes laboratoriais com diferentes tipos de sinais (sinais sinusoidais e sinais quadrados), gerados a partir de um gerador de funções. As medições permitiram analisar as funcionalidades do sistema, avaliar a sua capacidade de leitura e representação gráfica dos sinais, e identificar a necessidade de calibração para melhorar a precisão dos resultados.

Após a calibração, foi possível verificar uma melhoria na correspondência entre os valores medidos e os valores da simulação.

Em resumo, o trabalho cumpriu os objetivos definidos: foi possível construir um osciloscópio funcional com base num sistema embebido, realizar medições laboratoriais com sucesso e compreender a importância da calibração e do desenvolvimento cuidadoso de soluções de instrumentação. O projeto mostrou-se uma abordagem prática eficaz para a introdução à aquisição de dados e análise de sinais em contexto académico.

Programa principal (main.py)

```

1 <import T_Display
2 import math
3 import gc
4
5 # --- Constantes e escalas ---
6 WIDTH, HEIGHT = 240, 135 # Dimens es do ecr
7 GRID_HEIGHT = HEIGHT - 16 # Altura til para o
   gr fico (exclui t tulo)
8 XDIV, YDIV = 10, 6 # N de divis es
   horizontais e verticais
9 SCALES_TIME = [5, 10, 20, 50] # Escalas horizontais (
   tempo) em ms/div
10 SCALES_VOLT = [1, 2, 5, 10] # Escalas verticais (
   tens o) em V/div
11 SCALES_SPECTRUM_VOLT = [0.5, 1, 2.5, 5] # Escalas verticais para o
   espetro
12 SCALES_SPECTRUM_FREQ = [240, 120, 60, 24] # Escalas horizontais para
   o espetro (Hz/div)
13
14 # --- Vari veis de estado ---
15 tft = T_Display.TFT() # Instancia o display
16 scale_h = 1 # ndice da escala horizontal (default:
   10ms/div)
17 scale_v = 2 # ndice da escala vertical (default: 5V/
   div)
18 modo_espetro = False # Modo atual: False = tempo, True =
   espetro
19 index_spectrum_v = 0 # ndice da escala vertical do espetro
20 index_spectrum_h = 0 # ndice da escala horizontal do espetro
21
22 # --- Filtro Chebyshev IIR de 1 ordem (passa-banda centrado nos 100
   Hz) ---
23 def filtro_chebyshev(tensoes):
24     fs = 1000 / ((SCALES_TIME[scale_h] * 10) / WIDTH) # Frequ ncia
   de amostragem
25     f0 = 100 # Frequ ncia central do filtro (Hz)
26     BW = 50 # Largura de banda (Hz)
27     Q = f0 / BW
28     w0 = 2 * math.pi * f0 / fs
29     alpha = math.sin(w0) / (2 * Q)
30
31     # Coeficientes do filtro (normalizados)
32     b0 = alpha
33     b1 = 0
34     b2 = -alpha
35     a0 = 1 + alpha

```

```

36     a1 = -2 * math.cos(w0)
37     a2 = 1 - alpha
38
39     # Normaliza o
40     b0 /= a0
41     b1 /= a0
42     b2 /= a0
43     a1 /= a0
44     a2 /= a0
45
46     # Aplica o do filtro (f rmula da diferen a)
47     y = [0] * len(tensoes)
48     for i in range(2, len(tensoes)):
49         y[i] = b0 * tensoes[i] + b1 * tensoes[i-1] + b2 * tensoes[i-2]
50         - a1 * y[i-1] - a2 * y[i-2]
51     return y
52
53 # --- Convers o de ADC para tens o em Volts ---
54 def adc_para_volt(adc):
55     return 0.01265 * adc - 26.25898
56
57 # --- Leitura de amostras do ADC e convers o para tens o ---
58 def ler_amostras():
59     intervalo = SCALES_TIME[scale_h] * XDIV # Intervalo total (ms)
60     adc = tft.read_adc(WIDTH, intervalo) # L WIDTH amostras
61     durante esse intervalo
62     return [adc_para_volt(v) for v in adc] # Converte todas as
63     amostras
64
65 # --- Desenha a grelha e escalas no display ---
66 def mostrar_grelha():
67     tft.display_set(tft.BLACK, 0, 0, WIDTH, HEIGHT)
68     tft.display_write_grid(0, 16, WIDTH, HEIGHT-16, XDIV, YDIV, True,
69         tft.GREY1, tft.GREY2)
70     tft.display_write_str(tft.Arial16, f"{SCALES_VOLT[scale_v]}V/div",
71         5, HEIGHT - 135)
72     tft.display_write_str(tft.Arial16, f"{SCALES_TIME[scale_h]}ms/div",
73         120, HEIGHT - 135)
74     tft.set_wifi_icon(WIDTH - 16, HEIGHT - 135)
75
76 # --- Desenha a forma de onda no display ---
77 def desenhar_onda(tensoes, cor=tft.YELLOW):
78     y = [min(max(int((GRID_HEIGHT/2 + (GRID_HEIGHT/(YDIV*SCALES_VOLT[
79         scale_v]))*v), 0), GRID_HEIGHT-1)+16 for v in tensoes]
80     x = list(range(WIDTH))
81     tft.display_nline(cor, x, y)
82
83 # --- Mostra as medi es estat sticas: Vmax, Vmin, Vavg, Vrms ---

```

```

77 def mostrar_medidas(tensoes):
78     Vmax = max(tensoes)
79     Vmin = min(tensoes)
80     Vavg = sum(tensoes)/len(tensoes)
81     Vrms = math.sqrt(sum(v**2 for v in tensoes)/len(tensoes))
82     tft.display_set(tft.BLACK, 0, 0, WIDTH, HEIGHT)
83     tft.display_write_str(tft.Arial16, f"Vmax: {Vmax:.2f}V", 10, 90)
84     tft.display_write_str(tft.Arial16, f"Vmin: {Vmin:.2f}V", 10, 70)
85     tft.display_write_str(tft.Arial16, f"Vavg: {Vavg:.2f}V", 10, 50)
86     tft.display_write_str(tft.Arial16, f"Vrms: {Vrms:.2f}V", 10, 30)
87
88 # --- Envia email com os dados e medidas ---
89 def enviar_email(tensoes):
90     dt = (SCALES_TIME[scale_h]*10)/WIDTH / 1000
91     corpo = f"Foram recolhidos {len(tensoes)} pontos em {dt*len(tensoes):.2f}s.\n"
92     corpo += f"Vmax = {max(tensoes):.2f}V, Vmin = {min(tensoes):.2f}V\n"
93     corpo += f"Vavg = {sum(tensoes)/len(tensoes):.2f}V, Vrms = {math.sqrt(sum(v**2 for v in tensoes)/len(tensoes)):.2f}V"
94     tft.send_mail(dt, tensoes, corpo, "lucia.m.delgado.gomes@tecnico.ulisboa.pt")
95
96 # --- Cálculo e visualização do espectro (DFT manual) ---
97 def mostrar_espectro_dft(tensoes):
98     N = len(tensoes)
99     X_k = [0.0] * N
100     X_ss_k = [0.0] * N
101     for k in range(N // 2):
102         real = 0
103         imag = 0
104         for n in range(N):
105             real += tensoes[n] * math.cos(2 * math.pi * k * n / N)
106             imag += tensoes[n] * -math.sin(2 * math.pi * k * n / N)
107         X_k[k] = math.sqrt(real**2 + imag**2)
108         X_ss_k[k] = X_k[k] / N if k == 0 else 2 * X_k[k] / N
109
110     fs = 1000 / ((SCALES_TIME[scale_h] * 10) / WIDTH)
111     hz_div = SCALES_SPECTRUM_FREQ[index_spectrum_h]
112     freq_max = hz_div * XDIV
113     freq_resolution = fs / N
114
115     x_aux, y_aux = [], []
116     for k in range(N // 2):
117         freq = k * freq_resolution
118         x = int((freq / freq_max) * WIDTH)
119         if x >= WIDTH:
120             continue

```

```

121     y = X_ss_k[k] * (GRID_HEIGHT / (SCALES_SPECTRUM_VOLT[
122         index_spectrum_v] * YDIV))
123     y = min((y, GRID_HEIGHT)) + 16
124     x_aux.append(x)
125     y_aux.append(round(y))
126
127     tft.display_set(tft.BLACK, 0, 0, WIDTH, HEIGHT)
128     tft.display_write_grid(0, 16, WIDTH, GRID_HEIGHT, XDIV, YDIV,
129         False, tft.GREY1, tft.GREY2)
130     tft.display_nline(tft.MAGENTA, x_aux, y_aux)
131     tft.display_write_str(tft.Arial16, f"{SCALES_SPECTRUM_VOLT[
132         index_spectrum_v]:.1f} V/div", 5, 0)
133     tft.display_write_str(tft.Arial16, f"{SCALES_SPECTRUM_FREQ[
134         index_spectrum_h]} Hz/div", 120, 0)
135     tft.set_wifi_icon(WIDTH - 16, 0)
136
137 # --- Execu o Inicial ---
138 mostrar_grelha()
139 tensoes = ler_amostras()
140 desenhar_onda(tensoes)
141
142 # --- Ciclo principal de intera o com os bot es ---
143 def ciclo():
144     global scale_v, scale_h, modo_espectro, tensoes, index_spectrum_v,
145         index_spectrum_h
146     while tft.working():
147         b = tft.readButton()
148         if b != tft.NOTHING:
149             print("Bot o:", b)
150             gc.collect()
151             if b == 11: # Reset do modo tempo
152                 modo_espectro = False
153                 scale_h = 1
154                 scale_v = 2
155                 mostrar_grelha()
156                 tensoes = ler_amostras()
157                 desenhar_onda(tensoes)
158             elif b == 12: # Envia email
159                 enviar_email(tensoes)
160             elif b == 13: # Filtro Chebyshev
161                 mostrar_grelha()
162                 tensoes = ler_amostras()
163                 desenhar_onda(tensoes, cor=tft.YELLOW)
164                 filtrada = filtro_chebyshev(tensoes)
165                 desenhar_onda(filtrada, cor=tft.CYAN)
166             elif b == 21: # Alternar escala vertical
167                 if modo_espectro:

```

```
163         index_spectrum_v = (index_spectrum_v + 1) % len(  
164             SCALES_SPECTRUM_VOLT)  
165     mostrar_espectro_dft(tensoes)  
166     else:  
167         scale_v = (scale_v + 1) % len(SCALES_VOLT)  
168         mostrar_grelha()  
169         desenhara_onda(tensoes)  
170     elif b == 22: # Alternar escala horizontal  
171         if modo_espectro:  
172             index_spectrum_h = (index_spectrum_h + 1) % len(  
173                 SCALES_SPECTRUM_FREQ)  
174             mostrar_espectro_dft(tensoes)  
175         else:  
176             scale_h = (scale_h + 1) % len(SCALES_TIME)  
177             mostrar_grelha()  
178             tensoes = ler_amostras()  
179             desenhara_onda(tensoes)  
180     elif b == 23: # Ativar modo espectro  
181         modo_espectro = True  
182         mostrar_espectro_dft(tensoes)  
  
ciclo()>
```